CP4 - DBS - JPA aplikace

Michal Hrouda a Lukáš Hrubec

v1.1
26. 4. 2026

Obsah

Úvod . . . 2

Pokrytí zadání . . . 3

Datový model . . . 4
- Many-to-Many vazba . . . 4
- Dědičnost . . . 5

DAO vrstva . . . 6
- Parametrizovaný dotaz . . . 6

Servisní vrstva - 5 užití . . . 8

CRUD pokrytí dědičnosti, M:N a transakce CP-4 . . . 9
- Dědičnost (`Osoba -> Zamestnanec`) . . . 9
- N:M vazba (`UcastniSe`) . . . 9
- Transakce z CP-4 . . . 10

Použití view, triggeru a indexu z CP-4 . . . 12
- View . . . 12
- Trigger . . . 12
- Index . . . 12

Mapa odevzdání . . . 13

Úvod

Tento report popisuje Java aplikaci, která tvoří odevzdávanou část CP-4. Aplikace je postavená nad databází z CP-3 a používá Spring Boot 3.2 + Spring Data JPA (Hibernate jako provider). Běží jako CLI nad PostgreSQL na školním serveru `slon.felk.cvut.cz`.

Spuštění: skript `start.sh` v kořeni repozitáře (vyžaduje Java 17 a aplikované DDL z `docs/cp3/DDL.sql` plus objekty z `docs/cp4/`). Po spuštění se objeví číselné menu s položkami 1-14, jednotlivé volby odpovídají užitím popsaným níže.

Zdrojové kódy jsou v `src/main/java/cz/fel/hroudmi5/` a jsou rozdělené do čtyř balíčků:

- `model/` - JPA entity (mapování tabulek)
- `dao/` - Spring Data repozitáře a ručně psaný `ZamestnanecDao` s `EntityManager`
- `service/` - servisní vrstva s transakcemi
- `dto/` - vstupní DTO a výstupní pohledy (records)
- `Application.java` - vstupní bod aplikace s interaktivním menu

Pokrytí zadání

Zadání CP-4 vyžaduje, aby aplikace nad databází CP-3 obsahovala:

Bod zadání	**Realizace**
Datový model odpovídající celé databázi	Balíček `model/` - 10 entit, 1:1 s tabulkami z `docs/cp3/DDL.sql`
Many-to-Many vazba	`model/UcastniSe.java` + `model/UcastniSeId.java` - asociační entita s atributem `role`
Dědičnost	`model/Osoba.java` (`@MappedSuperclass`) - `model/Zamestnanec.java`
DAO vrstva s parametrizovaným dotazem	`dao/ZamestnanecDao.java` (4 metody, 3 styly parametrizace)
Servisní vrstva s 5 užitími (zápisové)	`service/` - vytvoření zaměstnance, převod, docházka, přiřazení na projekt, výměna manažera
Pokrytí transakce z CP-4	`service/ManazerService.vymenManazera` - SERIALIZABLE

Vedle těchto bodů aplikace explicitně demonstruje, že CRUD operace (CREATE, READ, UPDATE, DELETE) probíhají i nad dědičností a M:N vazbou - tj. ne jen čtení, ale také vkládání, změna a mazání. Mapa je v sekci **CRUD pokrytí dědičnosti, M:N a transakce CP-4**.

Datový model

Všech 10 tabulek z `docs/cp3/DDL.sql` má vlastní JPA entitu v balíčku `model/`. Mapování drží přesně: názvy tabulek a sloupců jsou převzaté beze změn (např. `@Table(name = "Zamestnanec")`, `@Column(name = "id_zamestnanec")`), constrainty z DDL (NOT NULL, UNIQUE, délky) jsou propsané do anotací. Identifikátory v DB jsou v uvozovkách a CamelCase, proto je v `application.properties` zapnuté `globally_quoted_identifiers=true`.

Tabulka v DDL	Java entita
Oddeleni	`model/Oddeleni.java`
Pozice	`model/Pozice.java`
Projekt	`model/Projekt.java`
Zamestnanec	`model/Zamestnanec.java` (dědí z `Osoba`)
Telefon	`model/Telefon.java`
Adresa	`model/Adresa.java`
Manazer	`model/Manazer.java`
Externista	`model/Externista.java`
ZaznamDochazky	`model/ZaznamDochazky.java`
UcastniSe	`model/UcastniSe.java` + `UcastniSeId.java`

Many-to-Many vazba

M:N vazbu mezi `Zamestnanec` a `Projekt` modeluje asociační entita `UcastniSe` (`src/main/java/cz/fel/hroudmi5/model/UcastniSe.java`). Vazba má vlastní atribut `role`, proto není použita anotace `@ManyToMany` - ta na spojovací tabulce `@JoinTable` neumí nést dodatečný sloupec. Místo toho je modelovaná jako samostatná entita s kompozitním primárním klíčem (`@EmbeddedId UcastniSeId`) a dvěma `@ManyToOne` vazbami s `@MapsId`.

```java
@Entity
@Table(name = "UcastniSe")
public class UcastniSe {
    @EmbeddedId
    private UcastniSeId id = new UcastniSeId();

    @Column(name = "role", nullable = false)
    private String role;

    @ManyToOne(fetch = FetchType.LAZY, optional = false)
    @MapsId("idZamestnanec")
    @JoinColumn(name = "id_zamestnanec")
    private Zamestnanec zamestnanec;

    @ManyToOne(fetch = FetchType.LAZY, optional = false)
    @MapsId("idProjekt")
    @JoinColumn(name = "id_projekt")
    private Projekt projekt;
}
```

Kód 1: Asociační entita pro M:N - klíčové části UcastniSe.java

Dědičnost

Dědičnost je realizována přes abstraktní třídu `Osoba` v souboru `src/main/java/cz/fel/hroudmi5/model/Osoba.java`. Drží sdílené osobní atributy `jmeno`, `prijmeni`, `email`, je označená anotací `@MappedSuperclass` a entita `Zamestnanec` z ní dědí (`extends Osoba`).

Zvolili jsme strategii `@MappedSuperclass` místo `@Inheritance(strategy = JOINED)` z následujícího důvodu: tabulky `Manazer` a `Externista` v DDL z CP-3 mají vlastní samostatný primární klíč (`id_manazer`, resp. `id_externista` typu SERIAL) a vazbu na zaměstnance řeší přes oddělený sloupec `id_zamestnanec` s UNIQUE constraintem. JPA strategie `JOINED` ale vyžaduje, aby primární klíč podtřídy byl zároveň cizím klíčem na primární klíč rodiče (sdílené ID), což by si vyžádalo přepsat DDL. Modelujeme proto `Manazer` a `Externista` jako samostatné entity s 1:1 vazbou na `Zamestnance` (logicky to dává smysl: jde o roli, kterou zaměstnanec může mít, ne o jiný typ osoby), a dědičnost se uplatňuje na úrovni `Osoba -> Zamestnanec`.

```java
@MappedSuperclass
public abstract class Osoba {
    @Column(name = "jmeno", nullable = false, length = 100)
    private String jmeno;

    @Column(name = "prijmeni", nullable = false, length = 100)
    private String prijmeni;

    @Column(name = "email", nullable = false, length = 100, unique = true)
    private String email;
}

@Entity
@Table(name = "Zamestnanec")
public class Zamestnanec extends Osoba {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "id_zamestnanec")
    private Integer idZamestnanec;
    // ...
}
```

Kód 2: Dědičnost - sdílené atributy v rodičovské třídě

DAO vrstva

DAO vrstva je v balíčku `src/main/java/cz/fel/hroudmi5/dao/`. Kombinuje dvě cesty:

- **Spring Data repozitáře** (`*Repository.java`) - rozhraní dědící z `JpaRepository`, využívají odvozené metody i vlastní `@Query` (např. `ManazerRepository.deleteByOddeleni`, `DochazkaRepository.findOtevreny`, `UcastniSeRepository.findByProjekt`).
- **Ručně psaný `ZamestnanecDao`** v `dao/ZamestnanecDao.java` - používá `EntityManager` a obsahuje parametrizované dotazy ve třech různých stylech (požadavek zadání).

Parametrizovaný dotaz

`ZamestnanecDao` schválně demonstruje různé varianty parametrizace dotazu, přesně podle přednášky `lecture-10-jdbc-jpa.pdf` (sekce **ODBC - prepared statement** a **JPQL - using parameters**). Konkatenace řetězce do dotazu se v aplikaci nikde nepoužívá - vše prochází přes `setParameter`, takže nehrozí SQL injection.

Metoda	Typ dotazu	Parametrizace
`najdiNastoupenePo`	JPQL	pojmenované parametry (`:datum`, `:idOddeleni`)
`vyhledejDleJmena`	native SQL nad `Zamestnanec`	pojmenované (`:vzor`)
`kontaktyManazeru`	native SQL nad view CP-4	bez parametrů
`kontaktyManazeruProOddeleni`	native SQL nad view CP-4	poziční (`?1`)

```java
public List<Zamestnanec> najdiNastoupenePo(LocalDate datum, Integer idOddeleni)
{
    String jpql = "SELECT z FROM Zamestnanec z WHERE z.datumNastupu > :datum"
            + (idOddeleni != null ? " AND z.oddeleni.idOddeleni = :idOddeleni" :
"")
            + " ORDER BY z.datumNastupu";
    TypedQuery<Zamestnanec> q = em.createQuery(jpql, Zamestnanec.class);
    q.setParameter("datum", datum);
    if (idOddeleni != null) {
        q.setParameter("idOddeleni", idOddeleni);
    }
    return q.getResultList();
}
```

Kód 3: Parametrizovaný JPQL s pojmenovanými parametry - ZamestnanecDao.najdiNastoupenePo

```java
public List<Object[]> kontaktyManazeruProOddeleni(String vzorOddeleni) {
    Query q = em.createNativeQuery(
        "SELECT oddeleni, manazer, email, telefon " +
        "FROM v_kontaktni_seznam_manazeru " +
        "WHERE oddeleni ILIKE ?1");
    q.setParameter(1, "%" + vzorOddeleni + "%");
    return q.getResultList();
}
```

Kód 4: Parametrizovaný native SQL s pozičním parametrem - ZamestnanecDao.kontaktyManazeruProOddeleni

Servisní vrstva - 5 užití

Servisní vrstva je v balíčku `src/main/java/cz/fel/hroudmi5/service/`. Každá veřejná metoda je označená `@Transactional`. Pět vybraných užití zaměřených na zápisové operace, jak požaduje zadání:

#	Užití	Soubor a metoda	Menu
1	Vytvoření nového zaměstnance	`service/ZamestnanecService.vytvor`	1
2	Převedení zaměstnance na jiné oddělení	`service/ZamestnanecService.prevedNaOddeleni`	2
3	Evidence docházky (příchod / odchod)	`service/DochazkaService.prichod`, `odchod`	3
4	Přiřazení zaměstnance na projekt (M:N)	`service/ProjektService.priradZamestnance`	4
5	Výměna manažera (transakce CP-4)	`service/ManazerService.vymenManazera`	5

V interaktivním menu v `Application.java` odpovídají těmto užitím volby 1-5. Pro úplnost CRUD operací nad povinnými prvky zadání (dědičnost, M:N) jsou v menu k dispozici i další doplňkové operace - viz následující sekce.

CRUD pokrytí dědičnosti, M:N a transakce CP-4

Cvičící požaduje, aby aplikace mezi dotazy obsahovala demonstraci 1) dědičnosti, 2) N:M vztahu a 3) transakce z CP-4 - a to **nejen na čtení**, ale i pro **vkládání, změnu a mazání**. Mapa pokrytí:

Dědičnost (`Osoba -> Zamestnanec`)

CRUD	Soubor a metoda	Menu
CREATE	`service/ZamestnanecService.vytvor` - vytvoří `Zamestnanec`, nastaví zděděné `jmeno/prijmeni/email` z `Osoba`	1
READ	`dao/ZamestnanecDao.vyhledejDleJmena` - native SQL filtrující dle `jmeno/prijmeni` (atributy z rodiče)	7
UPDATE	`service/ZamestnanecService.upravKontakt` - mění zděděné `jmeno/prijmeni/email`	14
DELETE	`service/ZamestnanecService.smaz`	11

```java
@Transactional
public ZamestnanecView upravKontakt(Integer idZamestnance,
                                    String jmeno, String prijmeni, String email)
{
    Zamestnanec z = getEntity(idZamestnance);
    if (jmeno != null && !jmeno.isBlank()) z.setJmeno(jmeno);
    if (prijmeni != null && !prijmeni.isBlank()) z.setPrijmeni(prijmeni);
    if (email != null && !email.isBlank()) z.setEmail(email);
    return ZamestnanecView.from(zamestnanecRepository.save(z));
}
```

Kód 5: UPDATE zděděných atributů - ZamestnanecService.upravKontakt

N:M vazba (`UcastniSe`)

CRUD	Soubor a metoda	Menu
CREATE	`service/ProjektService.priradZamestnance` - vloží nový řádek do `UcastniSe` (asociační entita s atributem `role`)	4
READ	`dao/UcastniSeRepository.findByProjekt` (JPQL přes `UcastniSe`) volaný z `service/ProjektService.clenoveProjektu`	-
UPDATE	`service/ProjektService.zmenRoli` - UPDATE atributu `role` na M:N vazbě (to je důvod, proč není čistá `@ManyToMany`)	12
DELETE	`service/ProjektService.odeberZamestnance` - smaže záznam z `UcastniSe`	13

```java
@Transactional
public UcastView zmenRoli(Integer idZamestnance, Integer idProjektu, String
novaRole) {
    UcastniSeId pk = new UcastniSeId(idZamestnance, idProjektu);
    UcastniSe u = ucastniSeRepository.findById(pk)
            .orElseThrow(() -> new EntityNotFoundException(
                    "Zamestnanec " + idZamestnance + " nema ucast na projektu "
+ idProjektu));
    u.setRole(novaRole);
    return UcastView.from(ucastniSeRepository.save(u));
}
```

Kód 6: UPDATE M:N vazby - ProjektService.zmenRoli (mění atribut na asociační entitě)

Transakce z CP-4

Transakce z `docs/cp4/transaction.sql` je `DELETE` z tabulky `Manazer` (smaže původního manažera oddělení) plus `INSERT` nového záznamu, vše v izolaci `SERIALIZABLE` proti race conditions. Implementace je v `service/ManazerService.vymenManazera` (volba 5 v menu).

Mezi `DELETE` a `INSERT` voláme `em.flush()`, jinak by Hibernate poslal oba příkazy najednou až na konci transakce a `INSERT` by spadl na unique constraintu `unique_oddeleni` (na sloupci `id_oddeleni`).

```java
@Transactional(isolation = Isolation.SERIALIZABLE)
public ManazerView vymenManazera(VymenaManazeraDto dto) {
    Oddeleni oddeleni = oddeleniRepository.findById(dto.getIdOddeleni())
            .orElseThrow(() -> new EntityNotFoundException(
                    "Oddeleni " + dto.getIdOddeleni() + " nenalezeno"));

    Zamestnanec novy =
zamestnanecRepository.findById(dto.getIdNovehoZamestnance())
            .orElseThrow(() -> new EntityNotFoundException(
                    "Zamestnanec " + dto.getIdNovehoZamestnance() + "
nenalezen"));

    if (!novy.getOddeleni().getIdOddeleni().equals(oddeleni.getIdOddeleni())) {
        throw new IllegalArgumentException(
                "Novy manazer musi byt zamestnancem oddeleni, ktere ma ridit");
    }

    manazerRepository.deleteByOddeleni(oddeleni.getIdOddeleni());
    em.flush();   // jinak by INSERT nize spadl na unique_oddeleni

    Manazer novyManazer = new Manazer();
    novyManazer.setOddeleni(oddeleni);
    novyManazer.setZamestnanec(novy);
    novyManazer.setUrovenPravomoci(
            dto.getUrovenPravomoci() != null ? dto.getUrovenPravomoci() : 1);
    manazerRepository.save(novyManazer);

    return ManazerView.from(novyManazer);
}
```

Kód 7: Transakce výměny manažera - ManazerService.vymenManazera (SERIALIZABLE)

Validace navíc kontroluje, že nový manažer je opravdu zaměstnancem daného oddělení. FK constraint `Ref_Manazer_to_Zamestnanec` v DDL to sice vynucuje, ale v Javě tu chybu zachytíme dřív s lepší hláškou.

Použití view, triggeru a indexu z CP-4

View

Pohled `v_kontaktni_seznam_manazeru` z `docs/cp4/view.sql` je v aplikaci využit prostřednictvím dvou metod v `dao/ZamestnanecDao.java`. Aplikace nad pohledem volá native SQL přímo - pohled se **nereplikuje JOINem v Javě**, to je explicitní požadavek pro CP-4.

```
public List<Object[]> kontaktyManazeru() {
    Query q = em.createNativeQuery(
        "SELECT oddeleni, manazer, email, telefon FROM
v_kontaktni_seznam_manazeru");
    return q.getResultList();
}

public List<Object[]> kontaktyManazeruProOddeleni(String vzorOddeleni) {
    Query q = em.createNativeQuery(
        "SELECT oddeleni, manazer, email, telefon " +
        "FROM v_kontaktni_seznam_manazeru " +
        "WHERE oddeleni ILIKE ?1");
    q.setParameter(1, "%" + vzorOddeleni + "%");
    return q.getResultList();
}
```

Kód 8: Native SQL nad view CP-4 - ZamestnanecDao

Servisní obal je v `service/ManazerService.kontakty(...)`, výstup mapuje na DTO `dto/KontaktManazeraView`. V menu volba 10 - uživatel zadá filtr (název oddělení, prázdné = celý seznam) a aplikace vypíše tabulku s oddělením, jménem manažera, e-mailem a telefonem.

Trigger

Trigger `trg_kontrola_dochazky` z `docs/cp4/trigger.sql` se aktivuje při vkládání nového záznamu do `ZaznamDochazky`. V Javě se trigger uplatňuje implicitně - aplikace ho nevolá explicitně, ale pokus o opakovaný příchod téhož zaměstnance bez ukončení předchozího záznamu skončí chybou v PostgreSQL, kterou Hibernate převede na výjimku.

Zápis docházky řeší metoda `prichod` v `service/DochazkaService.java`. Výjimku, kterou trigger způsobí, zachytí `try/catch` v hlavní smyčce v `Application.run` a vypíše ji uživateli jako „Chyba: …“.

V menu volba 3 - pokud uživatel dvakrát po sobě zadá příchod stejnému zaměstnanci bez odchodu mezi tím, trigger zápis zablokuje.

Index

Index `idx_dochazka_zamestnanec` z `docs/cp4/index.sql` je čistě databázová optimalizace. V Javě se neprojeví explicitně, ale dotaz `DochazkaRepository.findByZamestnanec` (`dao/DochazkaRepository.java`) ho automaticky využije při hledání záznamů podle ID zaměstnance.

Mapa odevzdání

Stručný přehled, kde co najít:

Co	**Kde**
Datový model (10 entit)	`src/main/java/cz/fel/hroudmi5/model/`
M:N vazba s atributem `role`	`model/UcastniSe.java`, `model/UcastniSeId.java`
Dědičnost (`@MappedSuperclass`)	`model/Osoba.java -> model/Zamestnanec.java`
DAO s parametrizovaným dotazem	`dao/ZamestnanecDao.java` (4 metody, 3 styly)
Spring Data repozitáře	`dao/*Repository.java`
Servisní vrstva	`service/`
Užití 1 - vytvoření zaměstnance (CREATE + dědičnost)	`service/ZamestnanecService.vytvor` (menu 1)
Užití 2 - převod na oddělení (UPDATE)	`service/ZamestnanecService.prevedNaOddeleni` (menu 2)
Užití 3 - docházka (CREATE + trigger CP-4)	`service/DochazkaService.prichod` / `odchod` (menu 3)
Užití 4 - přiřazení na projekt (CREATE M:N)	`service/ProjektService.priradZamestnance` (menu 4)
Užití 5 - výměna manažera (transakce CP-4)	`service/ManazerService.vymenManazera` (menu 5)
SELECT s parametrem (dědičnost)	`dao/ZamestnanecDao.vyhledejDleJmena` (menu 7)
SELECT nad view CP-4	`dao/ZamestnanecDao.kontaktyManazeru*` (menu 10)
DELETE zaměstnance (kaskáda na M:N)	`service/ZamestnanecService.smaz` (menu 11)
UPDATE M:N (atribut `role`)	`service/ProjektService.zmenRoli` (menu 12)
DELETE M:N (odebrání z projektu)	`service/ProjektService.odeberZamestnance` (menu 13)
UPDATE zděděných atributů (`Osoba`)	`service/ZamestnanecService.upravKontakt` (menu 14)
Aplikace triggeru z CP-4	`service/DochazkaService.prichod` (implicitně, menu 3)
Konfigurace	`src/main/resources/application.properties`
Spouštěcí skript	`start.sh`